

CS3009 – Computer Network



Instructor: Ms. Alisha Abid

alisha.abid@nu.edu.pk

Fall 2025

Name: Kainat Zahra, Mehreen Fatima, Amama Sajjad

Roll number: 23f-0713, 23f-0611, 23f-6014

National University of Computer & Emerging Sciences

Department: Computer Science

Application Section

Application Design:

- The system follows a **Client-Server Architecture**.
- **Central Server:** Manages connections from all campus clients.
- **Campus Clients:** Each campus (Lahore, Karachi, Islamabad, Peshawar, Multan, CFD) runs a client application that connects to the central server.
- **Department Users:** Simulated users interact with the client via a console interface.

Custom Protocol:

- **TCP (Reliable):**
 - Used for authentication and direct messaging between campuses.
 - **Message format:** TO:<TargetCampus> DEPT:<TargetDept>MSG:<Message>
 - Server routes the message to the target campus.
- **UDP (Connectionless):**
 - Used for sending heartbeat status and broadcasting announcements.
 - **Heartbeat format:** STATUS:<CampusName>:Online:<UDPPort>
 - **Broadcast format:** BROADCAST:<Message>

Features:

- Campus client authentication.
- Direct messaging between campuses.
- Periodic UDP heartbeat to the server.
- System-wide UDP announcements.
- Console menu for sending messages and viewing connection status.

Instructions to Compile and Run:

1. **Compile Server on one terminal:**
2. `g++ server.cpp -std=c++17 -Wall -pthread -o server`
3. **Compile Client on another terminal:**
4. `g++ client.cpp -std=c++17 -Wall -pthread -o client`
5. **Run Server:**
6. `./server`
7. **Run Client:**

8. ./client <CampusName> <Password>

Example:

./client Lahore NU-LHR-123

IP Addressing Table

Device	IP Address	Role
Central Server	192.168.1.1	TCP/UDP Server
Islamabad	192.168.1.2	Campus Client
Lahore	192.168.1.3	Campus Client
Karachi	192.168.1.4	Campus Client
Peshawar	192.168.1.5	Campus Client
Multan	192.168.1.6	Campus Client
CFD	192.168.1.7	Campus Client

Routing Strategy:

- Used **static routing** for simplicity.
- Each campus knows the IP of the central server.
- Direct client-to-client communication is routed via the server.
- UDP broadcasts are sent using client UDP ports stored at the server.

Server.cpp:

```
#include <iostream>
#include <string>
#include <map>
#include <vector>
#include <thread>
#include <mutex>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define TCP_PORT 8080
#define UDP_PORT 8081
#define BUFFER_SIZE 4096

using namespace std;

// Structure to hold client information
struct CampusClient {
    int tcpSocket;
    string campusName;
    string lastStatus;
    time_t lastSeen;
    int udpPort;
    string ipAddress;
};

// Global data structures
map<string, CampusClient> connectedClients; // campusName -> client info
mutex clientsMutex;
mutex coutMutex;

// Thread-safe console output
void safePrint(const string& message) {
    lock_guard<mutex> lock(coutMutex);
    cout << message << endl;
}

// Validate campus credentials
bool authenticateClient(const string& authMessage, string& campusName) {
    // Expected format: "Campus:Lahore,Pass:NU-LHR-123"
    size_t campusPos = authMessage.find("Campus:");
    size_t passPos = authMessage.find(",Pass:");

    if (campusPos == string::npos || passPos == string::npos) {
```

```

        return false;
    }

    campusName = authMessage.substr(campusPos + 7, passPos - campusPos - 7);
    string password = authMessage.substr(passPos + 6);

    // Hard-coded credentials for different campuses
    map<string, string> validCredentials = {
        {"Islamabad", "NU-ISB-123"},
        {"Lahore", "NU-LHR-123"},
        {"Karachi", "NU-KHI-123"},
        {"Peshawar", "NU-PSH-123"},
        {"Multan", "NU-MLT-123"},
        {"CFD", "NU-CFD-123"}
    };

    if (validCredentials.find(campusName) != validCredentials.end() &&
        validCredentials[campusName] == password) {
        return true;
    }

    return false;
}

// Handle individual campus client communication
void handleCampusClient(int clientSocket, string campusName) {
    char buffer[BUFFER_SIZE];

    while (true) {
        memset(buffer, 0, BUFFER_SIZE);
        int bytesReceived = recv(clientSocket, buffer, BUFFER_SIZE - 1, 0);

        if (bytesReceived <= 0) {
            safePrint("[SERVER] Campus " + campusName + " disconnected.");

            // Remove from connected clients
            {
                lock_guard<mutex> lock(clientsMutex);
                connectedClients.erase(campusName);
            }

            close(clientSocket);
            return;
        }

        string message(buffer);
        safePrint("[SERVER] Received from " + campusName + ": " + message);
    }
}

```

```

        // Parse message format:
        "TO:TargetCampus|DEPT:TargetDept|MSG:ActualMessage"
        size_t toPos = message.find("TO:");
        size_t deptPos = message.find("|DEPT:");
        size_t msgPos = message.find("|MSG:");

        if (toPos != string::npos && deptPos != string::npos && msgPos !=
string::npos) {
            string targetCampus = message.substr(toPos + 3, deptPos - toPos - 3);
            string targetDept = message.substr(deptPos + 6, msgPos - deptPos - 6);
            string actualMessage = message.substr(msgPos + 5);

            // Route message to target campus
            {
                lock_guard<mutex> lock(clientsMutex);
                if (connectedClients.find(targetCampus) != connectedClients.end())
                {
                    string forwardMsg = "FROM:" + campusName + "|DEPT:" +
targetDept + "|MSG:" + actualMessage;
                    send(connectedClients[targetCampus].tcpSocket,
forwardMsg.c_str(), forwardMsg.length(), 0);
                    safePrint("[SERVER] Routed message from " + campusName + " to
" + targetCampus);
                } else {
                    string errorMsg = "ERROR:Campus " + targetCampus + " is not
online.";
                    send(clientSocket, errorMsg.c_str(), errorMsg.length(), 0);
                    safePrint("[SERVER] Failed to route: " + targetCampus + " is
offline.");
                }
            }
        }
    }
}

// Handle UDP status updates
void handleUDPStatus(int udpSocket) {
    struct sockaddr_in clientAddr;
    socklen_t clientLen = sizeof(clientAddr);
    char buffer[BUFFER_SIZE];

    while (true) {
        memset(buffer, 0, BUFFER_SIZE);
        int bytesReceived = recvfrom(udpSocket, buffer, BUFFER_SIZE - 1, 0,
(struct sockaddr*)&clientAddr, &clientLen);

        if (bytesReceived > 0) {
            string statusMsg(buffer);

```

```

// Expected format: "STATUS:CampusName:Online:UDPPort"
size_t pos1 = statusMsg.find(":");
size_t pos2 = statusMsg.find(":", pos1 + 1);
size_t pos3 = statusMsg.find(":", pos2 + 1);

if (pos1 != string::npos && pos2 != string::npos) {
    string campusName = statusMsg.substr(pos1 + 1, pos2 - pos1 - 1);
    string status = statusMsg.substr(pos2 + 1, pos3 - pos2 - 1);
    int clientPort = 0;
    if (pos3 != string::npos) {
        clientPort = stoi(statusMsg.substr(pos3 + 1));
    }

    lock_guard<mutex> lock(clientsMutex);
    if (connectedClients.find(campusName) != connectedClients.end()) {
        connectedClients[campusName].lastStatus = status;
        connectedClients[campusName].lastSeen = time(nullptr);
        connectedClients[campusName].udpPort = clientPort;

        // Store client IP address
        char clientIP[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &(clientAddr.sin_addr), clientIP,
INET_ADDRSTRLEN);
        connectedClients[campusName].ipAddress = string(clientIP);

        safePrint("[UDP] Status update from " + campusName + ": " +
status);
    }
}

}

}

}

}

// Admin console for broadcasting messages
void adminConsole(int udpSocket) {
    sleep(2); // Let server initialize

    cout << "\n===== " << endl;
    cout << "      ADMIN CONSOLE - Central Server      " << endl;
    cout << "===== " << endl;

    while (true) {
        cout << "\nAdmin Menu: " << endl;
        cout << "1. View Connected Campuses " << endl;
        cout << "2. Broadcast Announcement (UDP) " << endl;
        cout << "3. Exit " << endl;
        cout << "Choice: ";
    }
}

```

```

int choice;
cin >> choice;
cin.ignore();

if (choice == 1) {
    lock_guard<mutex> lock(clientsMutex);
    cout << "\n----- Connected Campuses -----" << endl;
    if (connectedClients.empty()) {
        cout << "No campuses connected." << endl;
    } else {
        for (const auto& [campus, client] : connectedClients) {
            cout << "Campus: " << campus
                << " | Status: " << client.lastStatus
                << " | Last Seen: " << (time(nullptr) - client.lastSeen)
                << "s ago" << endl;
        }
        cout << "-----" << endl;
    }
} else if (choice == 2) {
    cout << "Enter announcement message: ";
    string announcement;
    getline(cin, announcement);

    string broadcastMsg = "BROADCAST:" + announcement;

    lock_guard<mutex> lock(clientsMutex);
    for (const auto& [campus, client] : connectedClients) {
        if (client.udpPort > 0 && !client.ipAddress.empty()) {
            struct sockaddr_in clientAddr;
            clientAddr.sin_family = AF_INET;
            clientAddr.sin_port = htons(client.udpPort);
            inet_pton(AF_INET, client.ipAddress.c_str(),
&clientAddr.sin_addr);

            sendto(udpSocket, broadcastMsg.c_str(), broadcastMsg.length(),
0,
                (struct sockaddr*)&clientAddr, sizeof(clientAddr));
        }
    }

    cout << "Broadcast sent to all campuses." << endl;
} else if (choice == 3) {
    cout << "Exiting admin console..." << endl;
    break;
}
}

```



```

}

int main() {
    // Create TCP socket
    int tcpSocket = socket(AF_INET, SOCK_STREAM, 0);
    if (tcpSocket < 0) {
        cerr << "Failed to create TCP socket" << endl;
        return 1;
    }

    // Set socket options
    int opt = 1;
    setsockopt(tcpSocket, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    // Bind TCP socket
    struct sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(TCP_PORT);

    if (bind(tcpSocket, (struct sockaddr*)&serverAddr, sizeof(serverAddr)) < 0) {
        cerr << "TCP bind failed" << endl;
        return 1;
    }

    // Listen on TCP socket
    if (listen(tcpSocket, 10) < 0) {
        cerr << "Listen failed" << endl;
        return 1;
    }

    // Create UDP socket
    int udpSocket = socket(AF_INET, SOCK_DGRAM, 0);
    if (udpSocket < 0) {
        cerr << "Failed to create UDP socket" << endl;
        return 1;
    }

    // Bind UDP socket
    struct sockaddr_in udpAddr;
    udpAddr.sin_family = AF_INET;
    udpAddr.sin_addr.s_addr = INADDR_ANY;
    udpAddr.sin_port = htons(UDP_PORT);

    if (bind(udpSocket, (struct sockaddr*)&udpAddr, sizeof(udpAddr)) < 0) {
        cerr << "UDP bind failed" << endl;
        return 1;
    }
}

```

```

cout << "===== " << endl;
cout << "  NU-Information Exchange System" << endl;
cout << "      Central Server Started" << endl;
cout << "===== " << endl;
cout << "TCP Port: " << TCP_PORT << endl;
cout << "UDP Port: " << UDP_PORT << endl;
cout << "Waiting for campus connections..." << endl;
cout << "===== \n" << endl;

// Start UDP status handler thread
thread udpThread(handleUDPStatus, udpSocket);
udpThread.detach();

// Start admin console thread
thread adminThread(adminConsole, udpSocket);
adminThread.detach();

// Accept client connections
while (true) {
    struct sockaddr_in clientAddr;
    socklen_t clientLen = sizeof(clientAddr);

    int clientSocket = accept(tcpSocket, (struct sockaddr*)&clientAddr,
&clientLen);
    if (clientSocket < 0) {
        cerr << "Accept failed" << endl;
        continue;
    }

    // Receive authentication message
    char authBuffer[BUFFER_SIZE];
    memset(authBuffer, 0, BUFFER_SIZE);
    recv(clientSocket, authBuffer, BUFFER_SIZE - 1, 0);

    string campusName;
    if (authenticateClient(string(authBuffer), campusName)) {
        // Authentication successful
        string response = "AUTH_SUCCESS";
        send(clientSocket, response.c_str(), response.length(), 0);

        // Add to connected clients
        {
            lock_guard<mutex> lock(clientsMutex);
            connectedClients[campusName] = {clientSocket, campusName,
"Online", time(nullptr), 0, ""};
        }
    }
}

```

```

        safePrint("[SERVER] Campus " + campusName + " connected and
authenticated.");

        // Handle client in a new thread
        thread clientThread(handleCampusClient, clientSocket, campusName);
        clientThread.detach();
    } else {
        // Authentication failed
        string response = "AUTH_FAILED";
        send(clientSocket, response.c_str(), response.length(), 0);
        close(clientSocket);
        safePrint("[SERVER] Authentication failed for a client.");
    }
}

close(tcpSocket);
close(udpSocket);
return 0;
}

```

Client.cpp:

```

#include <iostream>
#include <string>
#include <thread>
#include <mutex>
#include <vector>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <chrono>

#define SERVER_IP "127.0.0.1"
#define TCP_PORT 8080
#define UDP_PORT 8081
#define UDP_BASE_PORT 9000
#define BUFFER_SIZE 4096

using namespace std;

string campusName;
string campusPassword;
int tcpSocket;
int udpSocket;
int udpListenPort;
mutex coutMutex;

```

```

bool isConnected = false;
vector<string> inboxMessages;

// Thread-safe console output
void safePrint(const string& message) {
    lock_guard<mutex> lock(coutMutex);
    cout << message << endl;
}

// Listen for incoming TCP messages from server
void receiveTCPMessages() {
    char buffer[BUFFER_SIZE];

    while (isConnected) {
        memset(buffer, 0, BUFFER_SIZE);
        int bytesReceived = recv(tcpSocket, buffer, BUFFER_SIZE - 1, 0);

        if (bytesReceived <= 0) {
            safePrint("\n[ERROR] Connection to server lost!");
            isConnected = false;
            break;
        }

        string message(buffer);

        if (message.find("ERROR:") == 0) {
            safePrint("\n[SERVER ERROR] " + message.substr(6));
        } else {
            size_t fromPos = message.find("FROM:");
            size_t deptPos = message.find("|DEPT:");
            size_t msgPos = message.find("|MSG:");

            if (fromPos != string::npos && deptPos != string::npos && msgPos
!= string::npos) {
                string fromCampus = message.substr(fromPos + 5, deptPos -
fromPos - 5);
                string department = message.substr(deptPos + 6, msgPos -
deptPos - 6);
                string actualMessage = message.substr(msgPos + 5);

                string formattedMsg;
                formattedMsg +=
"\n|-----|\n";
                formattedMsg += "    NEW MESSAGE
RECEIVED    |\n";
                formattedMsg +=
"|-----|\n";
            }
        }
    }
}

```

```

        formattedMsg += "|| From Campus: " + fromCampus
        ||\n";
        formattedMsg += "|| Department: " + department
        ||\n";
        formattedMsg += "|| Message: " + actualMessage
        ||\n";
        formattedMsg +=
        "||=====||\n";

        safePrint(formattedMsg);

        lock_guard<mutex> lock(coutMutex);
        inboxMessages.push_back(formattedMsg);
    }
}

// Send UDP heartbeat status to server
void sendHeartbeat() {
    struct sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(UDP_PORT);
    inet_pton(AF_INET, SERVER_IP, &serverAddr.sin_addr);

    while (isConnected) {
        string statusMsg = "STATUS:" + campusName + ":Online:" +
to_string(udpListenPort);
        sendto(udpSocket, statusMsg.c_str(), statusMsg.length(), 0,
        (struct sockaddr*)&serverAddr, sizeof(serverAddr));
        this_thread::sleep_for(chrono::seconds(10));
    }
}

// Listen for UDP broadcasts from server
void receiveUDPBroadcasts() {
    int udpListenSocket = socket(AF_INET, SOCK_DGRAM, 0);
    if (udpListenSocket < 0) {
        safePrint("[ERROR] Failed to create UDP listen socket");
        return;
    }

    int opt = 1;
    setsockopt(udpListenSocket, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in listenAddr;
    listenAddr.sin_family = AF_INET;
    listenAddr.sin_addr.s_addr = INADDR_ANY;

```

```

listenAddr.sin_port = htons(udpListenPort);

int attempts = 0;
while (bind(udpListenSocket, (struct sockaddr*)&listenAddr,
sizeof(listenAddr)) < 0 && attempts < 10) {
    udpListenPort++;
    listenAddr.sin_port = htons(udpListenPort);
    attempts++;
}

if (attempts >= 10) {
    safePrint("[ERROR] Failed to bind UDP listen socket after 10
attempts");
    close(udpListenSocket);
    return;
}

safePrint("[INFO] UDP listening on port " + to_string(udpListenPort));

char buffer[BUFFER_SIZE];
struct sockaddr_in serverAddr;
socklen_t serverLen = sizeof(serverAddr);

while (isConnected) {
    memset(buffer, 0, BUFFER_SIZE);
    int bytesReceived = recvfrom(udpListenSocket, buffer, BUFFER_SIZE - 1,
0,
                                (struct sockaddr*)&serverAddr,
&serverLen);

    if (bytesReceived > 0) {
        string message(buffer);
        if (message.find("BROADCAST:") == 0) {
            string announcement = message.substr(10);
            safePrint("\n[SYSTEM-WIDE ANNOUNCEMENT]");
            safePrint("[ " + announcement + " ]");
            safePrint("\n");
        }
    }
}

close(udpListenSocket);
}

// Display message inbox
void viewInbox() {

```

```

lock_guard<mutex> lock(coutMutex);
if (inboxMessages.empty()) {
    cout << "\n[INFO] No messages received yet.\n";
    return;
}

cout << "\n===== RECEIVED MESSAGES =====\n";
for (const auto& msg : inboxMessages) {
    cout << msg << endl;
}
cout << "=====\n";
}

// Display main menu and handle user interaction
void displayMenu() {
    while (isConnected) {
        cout << "\n" << endl;
        cout << "||    NU-Information Exchange System    ||" << endl;
        cout << "||    Campus:" << campusName << string(30 -
campusName.length(), ' ') << "||" << endl;
        cout << "||" << endl;
        cout << "||  1. Send Message to Another Campus  ||" << endl;
        cout << "||  2. View Received Messages         ||" << endl;
        cout << "||  3. View Connection Status         ||" << endl;
        cout << "||  4. Exit                           ||" << endl;
        cout << "||" << endl;
        cout << "Enter your choice: ";

        int choice;
        cin >> choice;
        cin.ignore();

        if (choice == 1) {
            viewInbox();

            cout << "\nAvailable Campuses: Islamabad, Lahore, Karachi,
Peshawar, Multan, CFD" << endl;
            cout << "Enter target campus name: ";
            string targetCampus;
            getline(cin, targetCampus);

            cout << "Available Departments: Admissions, Academics, IT, Sports"
<< endl;
            cout << "Enter target department: ";
            string targetDept;
            getline(cin, targetDept);

            cout << "Enter your message: ";

```

```

        string message;
        getline(cin, message);

        string formattedMsg = "TO:" + targetCampus + "|DEPT:" + targetDept
+ "|MSG:" + message;

        send(tcpSocket, formattedMsg.c_str(), formattedMsg.length(), 0);
        cout << "\n[SUCCESS] Message sent to " << targetCampus << " (" <<
targetDept << ")" << endl;
    }
    else if (choice == 2) {
        viewInbox();
    }
    else if (choice == 3) {
        cout << "\n" << endl;
        cout << "    Connection Status    " << endl;
        cout << "    " << endl;
        cout << "    Campus: " << campusName << "    " << endl;

        cout << "    Status: Connected to Central Server    " << endl;
        cout << "    TCP: Active    " << endl;
        cout << "    UDP Heartbeat: Active    " << endl;
        cout << "    UDP Listen Port: " << udpListenPort <<
        "    " << endl;
        cout << "    " << endl;
    }
    else if (choice == 4) {
        cout << "\nDisconnecting from server..." << endl;
        isConnected = false;
        close(tcpSocket);
        close(udpSocket);
        cout << "Goodbye!" << endl;
        exit(0);
    }
    else {
        cout << "\n[ERROR] Invalid choice. Please try again." << endl;
    }
}

int main(int argc, char* argv[]) {
    if (argc != 3) {
        cout << "Usage: ./client <CampusName> <Password>" << endl;
        cout << "Example: ./client Lahore NU-LHR-123" << endl;
        return 1;
    }

    campusName = argv[1];

```



```

campusPassword = argv[2];

int portOffset = 0;
for (char c : campusName) portOffset += c;
udpListenPort = UDP_BASE_PORT + (portOffset % 100);

tcpSocket = socket(AF_INET, SOCK_STREAM, 0);
if (tcpSocket < 0) {
    cerr << "Failed to create TCP socket" << endl;
    return 1;
}

struct sockaddr_in serverAddr;
serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(TCP_PORT);
inet_pton(AF_INET, SERVER_IP, &serverAddr.sin_addr);

cout << "Connecting to Central Server..." << endl;
if (connect(tcpSocket, (struct sockaddr*)&serverAddr, sizeof(serverAddr))
< 0) {
    cerr << "Failed to connect to server" << endl;
    return 1;
}

string authMsg = "Campus:" + campusName + ",Pass:" + campusPassword;
send(tcpSocket, authMsg.c_str(), authMsg.length(), 0);

char buffer[BUFFER_SIZE];
memset(buffer, 0, BUFFER_SIZE);
recv(tcpSocket, buffer, BUFFER_SIZE - 1, 0);

string response(buffer);
if (response == "AUTH_SUCCESS") {
    cout << "\n" << endl;
    cout << "    Authentication Successful    " << endl;
    cout << "    Connected to Central Server    " << endl;
    cout << "\n" << endl;

    isConnected = true;
    udpSocket = socket(AF_INET, SOCK_DGRAM, 0);

    thread tcpReceiveThread(receiveTCPMessages);
tcpReceiveThread.detach();
    thread udpHeartbeatThread(sendHeartbeat); udpHeartbeatThread.detach();
    thread udpBroadcastThread(receiveUDPBroadcasts);
udpBroadcastThread.detach();

    sleep(1);
}

```

```

        displayMenu();
    } else {
        cout << "\n[ERROR] Authentication failed! Invalid campus name or
password." << endl;
        close(tcpSocket);
        return 1;
    }

    return 0;
}

```

Output:

Interface of server:

```

ns3@ns3-virtual-machine: ~/Desktop/cnlabproj
ns3@ns3-virtual-machine:~/Desktop/cnlabproj$ ./server
=====
  NU-Information Exchange System
  Central Server Started
=====
TCP Port: 8080
UDP Port: 8081
Waiting for campus connections...
=====

=====
  ADMIN CONSOLE - Central Server
=====

Admin Menu:
1. View Connected Campuses
2. Broadcast Announcement (UDP)
3. Exit
Choice: [SERVER] Campus Lahore connected and authenticated.
[UDP] Status update from Lahore: Online
[UDP] Status update from Lahore: Online
[UDP] Status update from Lahore: Online

```

Interface for client:

```

ns3@ns3-virtual-machine:~/Desktop/cnlabproj$ ./client CFD NU-CFD-123
Connecting to Central Server...

Authentication Successful!
Connected to Central Server

[INFO] UDP listening on port 9005

NU-Information Exchange System
Campus:CFD

1. Send Message to Another Campus
2. View Received Messages
3. View Connection Status
4. Exit

```

Choice #01 send message:

```
Enter your choice: 1
[INFO] No messages received yet.
Available Campuses: Islamabad, Lahore, Karachi, Peshawar, Multan, CFD
Enter target campus name: Lahore
Available Departments: Admissions, Academics, IT, Sports
Enter target department: IT
Enter your message: Resolve student internet messages
[SUCCESS] Message sent to Lahore (IT)
```

Central server receives and sends the message target campus:

```
[UDP] Status update from Lahore: Online
[SERVER] Received from Lahore: TO:Lahore|DEPT:IT|MSG:Resolve student internet messages
[SERVER] Routed message from Lahore to Lahore
[UDP] Status update from Lahore: Online
```

Choice #02 show received by the target campus:

```
===== RECEIVED MESSAGES =====

+-----+
| NEW MESSAGE RECEIVED |
+-----+
| From Campus: Lahore   |
| Department: IT       |
| Message: Resolve student internet messages |
+-----+
```

Choice #03 Connection status:

```
Enter your choice: 3

+-----+
| Connection Status    |
+-----+
| Campus: Lahore       |
| Status: Connected to Central Server |
| TCP: Active          |
| UDP Heartbeat: Active |
| UDP Listen Port: 9003 |
+-----+
```

Choice #04 exit with real time connection status message

```
Enter your choice: 4
Disconnecting from server...
Goodbye!
```

Real time connection status message (client message)

```
[UDP] Status update from Lahore: Online
[UDP] Status update from Lahore: Online
[UDP] Status update from Lahore: Online
[SERVER] Campus Lahore disconnected.
```

Broadcasting message:

```
Choice: [UDP] Status update from CFD: Online
2
Enter announcement message: All the unive[UDP] Status update from CFD: Online
rsities are requested to send documents
Broadcast sent to all campuses.
```

Broadcast received by the client campus:



